



Sistemas Embebidos I

Otoño 2026

Práctica #5: HW timers: compare/capture; timer-based debounces

María Fernanda Conde Calatayud

Andrea Ceja Lara

07/09/26

Índice

<i>Práctica #5: HW timers: compare/capture; timer-based debounces</i>	<i>2</i>
Introducción	2
Descripción de la práctica	2
Objetivo	2
Marco teórico	2
Desarrollo	3
Materiales	3
Esquemático	4
Desarrollo.....	5
Resultados	7
Evidencias	8
Conclusiones	9

Práctica #5: HW timers: compare/capture; timer-based debounces

Introducción

Descripción de la práctica

La práctica consta de dos etapas:

1. **Frecuencímetro por marcas de tiempo (*Timestamping*):** Mide el periodo, la frecuencia y el *duty cycle* de una señal externa registrando los tiempos de sus flancos (subida y bajada) mediante una interrupción de GPIO. Incluye el monitoreo de la rutina con un pin de traza, la evaluación en distintas frecuencias y el cálculo del presupuesto de error y su frecuencia máxima.
2. **Antirrebote por temporizador (*Timer-based debounce*):** Filtra los rebotes de un botón mecánico sin bloquear la CPU. Al detectar el primer flanco, deshabilita la interrupción de GPIO y activa una alarma *one-shot* por el tiempo de rebote medido; al expirar la alarma, confirma el estado del pin, ejecuta la acción correspondiente y rehabilita la interrupción.

Objetivo

Desarrollar habilidades en la medición precisa de señales temporales y la gestión de entradas mecánicas en un microcontrolador (RP2350) mediante el uso de temporizadores de hardware, interrupciones de GPIO y rutinas de atención eficientes.

Marco teórico

Generación vs. Medición de Tiempo:

- **Generar tiempo (*Compare*):** El software establece un instante futuro en un registro de comparación; el hardware evalúa el contador interno e interrumpe cuando los valores coinciden (eventos internos/periódicos).
- **Medir tiempo (*Capture*):** Un evento externo fija el momento de ocurrencia. Se toma el valor del contador al instante del flanco para calcular diferencias temporales entre transiciones consecutivas.

Medición por marcas de tiempo (*Timestamping*):

A partir de tres marcas de tiempo consecutivas en los flancos de subida y bajada ($t_{subida1}$, t_{bajada} , $t_{subida2}$), se pueden derivar analíticamente los parámetros esenciales de una señal:

- **Periodo (T):** $T = t_{subida2} - t_{subida1}$
- **Tiempo en alto (t_{alto}):** $t_{alto} = t_{bajada} - t_{subida1}$
- **Frecuencia (f):** $f = \frac{1}{T}$
- **Ciclo de trabajo (*Duty Cycle*):** $duty = \frac{t_{alto}}{T} \times 100\%$

Presupuesto de error y Límite de Frecuencia:

El error en la medición se compone del error de resolución del *tick* (p. ej. $\pm 1 \mu s$ por lectura), el *jitter* en la latencia de las interrupciones y la interrupción por otras rutinas concurrentes. A medida que la frecuencia de la señal aumenta y su periodo se reduce, la incertidumbre fija de la resolución representa una fracción porcentual mayor del tiempo total, estableciendo un límite práctico superior para las mediciones por software.

Rebote mecánico y Antirrebote por Temporizador (*Debounce*):

Los contactos mecánicos de un pulsador generan una serie de transiciones o ruidos eléctricos (rebotes) que duran típicamente entre 1 ms y 20 ms antes de estabilizarse. En lugar de pausar el procesador con retardo activo (*sleep*), el antirrebote por *timer* utiliza un enfoque por eventos:

1. El primer flanco dispara una IRQ que deshabilita la línea de interrupción y agenda una alarma *one-shot* por un tiempo N ms.
2. Tras la ventana de tiempo, el *callback* de la alarma reevalúa la entrada física para confirmar que el estado se mantiene.
3. Si la lectura es válida, ejecuta la acción asociada y rehabilita la interrupción de GPIO para futuros eventos.

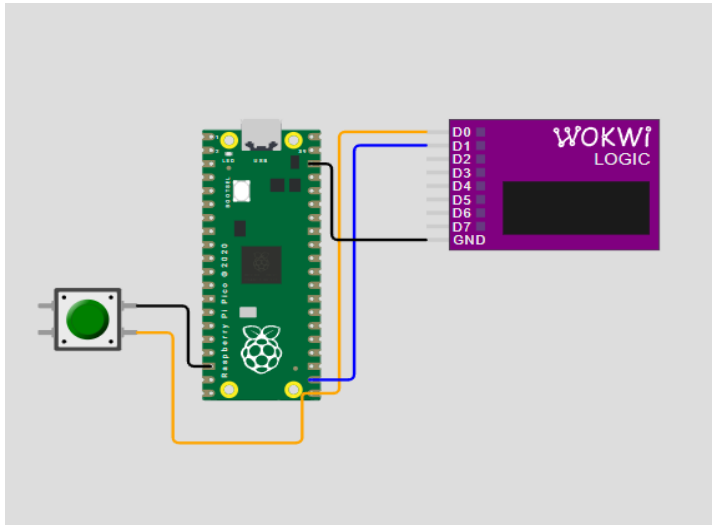
Desarrollo

Materiales

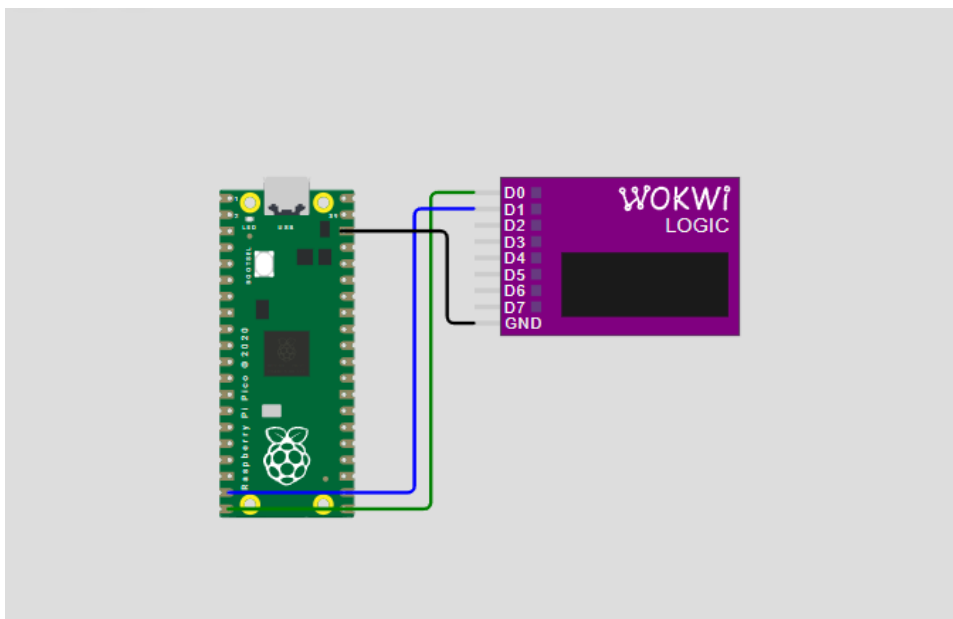
- Raspberry Pi Pico
- Protoboard
- Jumpers
- Simulador wowki
- Botón

Esquemático

Antirrebote



Frecuencia



Desarrollo

Frecuencia

```
#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/gpio.h"
#include "hardware/timer.h"
#include "hardware/pwm.h"
#include "hardware/structs/sio.h"

#define GEN_PIN 16 //pin generador pwm
#define SIG_PIN 15 //pin medidor de senal
#define TRAZA_PIN 14 //pin de traza isr

//cambiar segÃ³n la prueba: 100, 1000 o 10000
#define FREQ_TEST_HZ 10000

static volatile uint32_t t_subida_prev = 0;
static volatile uint32_t t_bajada = 0;
static volatile uint32_t T_ticks = 0;
static volatile uint32_t alto_ticks = 0;
static volatile bool listo = false;

static void sig_isr(uint gpio, uint32_t events) {
    uint32_t t = timer_hw->timerawl; //lee tiempo actual
    sio_hw->gpio_set = (1u << TRAZA_PIN); //sube traza
    if (events & GPIO_IRQ_EDGE_RISE) {
        if (t_subida_prev != 0) {
            T_ticks = t - t_subida_prev; //calcula periodo
            alto_ticks = t_bajada - t_subida_prev; //calcula tiempo en alto
            listo = true; //avisa dato listo
        }
        t_subida_prev = t;
    } else if (events & GPIO_IRQ_EDGE_FALL) {
        t_bajada = t; //guarda flanco de bajada
    }
    sio_hw->gpio_clr = (1u << TRAZA_PIN); //baja traza
    gpio_acknowledge_irq(gpio, events); //limpia interrupcion
}

static void iniciar_generador_pwm(uint pin, uint32_t freq_hz) {
    gpio_set_function(pin, GPIO_FUNC_PWM);
    uint slice = pwm_gpio_to_slice_num(pin);
    pwm_set_clkdiv(slice, 125.0f); //1 tick = 1 Åºs a 125 MHz
    uint32_t wrap = 1000000 / freq_hz;
    pwm_set_wrap(slice, (uint16_t)(wrap - 1));
    pwm_set_chan_level(slice, pwm_gpio_to_channel(pin), (uint16_t)(wrap / 2));
    pwm_set_enabled(slice, true);
}

int main(void) {
    stdio_init_all();
    gpio_init(TRAZA_PIN);
    gpio_set_dir(TRAZA_PIN, GPIO_OUT);
    gpio_put(TRAZA_PIN, 0);
    gpio_init(SIG_PIN);
    gpio_set_dir(SIG_PIN, GPIO_IN);
    gpio_disable_pulls(SIG_PIN);
    gpio_set_irq_enabled_with_callback(
        SIG_PIN,
        GPIO_IRQ_EDGE_RISE | GPIO_IRQ_EDGE_FALL, //escucha ambos flancos
        true,
```

```

        &sig_isr
    );
    iniciar_generador_pwm(GEN_PIN, FREQ_TEST_HZ);
    while (true) {
        if (listo) {
            uint32_t T = T_ticks; //copia local
            uint32_t alto = alto_ticks;
            listo = false;
            if (T > 0) {
                float f_hz = 1e6f / (float)T; //calcula frecuencia
                float duty = 100.0f * (float)alto / (float)T; //calcula duty
                printf("T = %u us | f = %.2f Hz | duty = %.1f %%\n", T, f_hz, duty);
            }
            sleep_ms(200);
        }
    }
    return 0;
}

```

Antirrebote

```

#include <stdio.h>
#include "pico/stdlib.h"
#include "hardware/gpio.h"
#include "hardware/timer.h"

#define BTN_PIN 16
#define LED_PIN 25 //led integrado en la placa
#define LED_MONITOR_PIN 17 //salida hacia D1 del analizador l gico
#define VENTANA_MS 20 //espera del debounce

static int64_t fin_debounce(alarm_id_t id, void *user_data) {
    if (gpio_get(BTN_PIN) == 0) { //verifica boton presionado
        gpio_xor_mask((1u << LED_PIN) | (1u << LED_MONITOR_PIN)); //conmuta leds
        printf("Boton confirmado -> LED conmutado\n");
    }
    gpio_set_irq_enabled(BTN_PIN, GPIO_IRQ_EDGE_FALL, true); //rehabilita interrupcion
    return 0; //one-shot
}

static void btn_isr(uint gpio, uint32_t events) {
    gpio_set_irq_enabled(BTN_PIN, GPIO_IRQ_EDGE_FALL, false); //apaga interrupcion
    add_alarm_in_ms(VENTANA_MS, fin_debounce, NULL, true); //arma temporizador
    gpio_acknowledge_irq(gpio, events); //limpia interrupcion
}

int main(void) {
    stdio_init_all();
    gpio_init(LED_PIN);
    gpio_set_dir(LED_PIN, GPIO_OUT);
    gpio_put(LED_PIN, 0);
    gpio_init(LED_MONITOR_PIN);
    gpio_set_dir(LED_MONITOR_PIN, GPIO_OUT);

    gpio_put(LED_MONITOR_PIN, 0);
    gpio_init(BTN_PIN);
    gpio_set_dir(BTN_PIN, GPIO_IN);
    gpio_pull_up(BTN_PIN); //activa pull up
    gpio_set_irq_enabled_with_callback(
        BTN_PIN,
        GPIO_IRQ_EDGE_FALL, //flanco de bajada
        true,
        &btn_isr
    );
    printf("Antirrebote listo. Presiona el pulsador.\n");
    while (true) {
        tight_loop_contents();
    }
    return 0;
}

```

Resultados

1. Cálculos de la Tabla del Frecuencímetro

Prueba a 100 Hz (del archivo VCD a 100 Hz):

- Flanco de subida 1: $t_{\text{subida1}} = 10\,063\,856\text{ ns} = 10\,063.856\text{ }\mu\text{s}$
- Flanco de bajada: $t_{\text{bajada}} = 15\,063\,856\text{ ns} = 15\,063.856\text{ }\mu\text{s}$
- Flanco de subida 2: $t_{\text{subida2}} = 20\,063\,872\text{ ns} = 20\,063.872\text{ }\mu\text{s}$

$$T = t_{\text{subida2}} - t_{\text{subida1}} = 20\,063.872 - 10\,063.856 = 10\,000.016\text{ }\mu\text{s} \approx 10\,000\text{ }\mu\text{s}$$

$$t_{\text{alto}} = t_{\text{bajada}} - t_{\text{subida1}} = 15\,063.856 - 10\,063.856 = 5\,000.000\text{ }\mu\text{s}$$

$$f_{\text{medida}} = \frac{10^6}{T} = \frac{1\,000\,000}{10\,000} = 100.00\text{ Hz}$$

$$\text{Duty} = \frac{t_{\text{alto}}}{T} \times 100\% = \frac{5\,000}{10\,000} \times 100\% = 50.0\%$$

$$\text{err. rel.} = \frac{|100.00 - 100|}{100} \times 100\% = 0.0\%$$

$$\text{err. duty} = |50.0\% - 50.0\%| = 0.0\%$$

Prueba a 1 kHz (del archivo VCD a 1 kHz):

- Flanco de subida 1: $t_{\text{subida1}} = 1\,061\,304\text{ ns} = 1\,061.304\text{ }\mu\text{s}$
- Flanco de bajada: $t_{\text{bajada}} = 1\,561\,304\text{ ns} = 1\,561.304\text{ }\mu\text{s}$
- Flanco de subida 2: $t_{\text{subida2}} = 2\,061\,320\text{ ns} = 2\,061.320\text{ }\mu\text{s}$

$$T = t_{\text{subida2}} - t_{\text{subida1}} = 2\,061.320 - 1\,061.304 = 1\,000.016\text{ }\mu\text{s} \approx 1\,000\text{ }\mu\text{s}$$

$$t_{\text{alto}} = t_{\text{bajada}} - t_{\text{subida1}} = 1\,561.304 - 1\,061.304 = 500.000\text{ }\mu\text{s}$$

$$f_{\text{medida}} = \frac{10^6}{T} = \frac{1\,000\,000}{1\,000} = 1\,000.00\text{ Hz}$$

$$\text{Duty} = \frac{500}{1\,000} \times 100\% = 50.0\%$$

$$\text{err. rel.} = \frac{|1\,000 - 1\,000|}{1\,000} \times 100\% = 0.0\%$$

$$\text{err. duty} = |50.0\% - 50.0\%| = 0.0\%$$

Prueba a 10 kHz (del archivo VCD a 10 kHz):

- Flanco de subida 1: $t_{\text{subida1}} = 161\,288\text{ ns} = 161.288\text{ }\mu\text{s}$
- Flanco de bajada: $t_{\text{bajada}} = 211\,288\text{ ns} = 211.288\text{ }\mu\text{s}$
- Flanco de subida 2: $t_{\text{subida2}} = 261\,304\text{ ns} = 261.304\text{ }\mu\text{s}$

$$T = t_{\text{subida2}} - t_{\text{subida1}} = 261.304 - 161.288 = 100.016\text{ }\mu\text{s} \approx 100\text{ }\mu\text{s}$$

$$t_{\text{alto}} = t_{\text{bajada}} - t_{\text{subida1}} = 211.288 - 161.288 = 50.000\text{ }\mu\text{s}$$

$$f_{\text{medida}} = \frac{10^6}{T} = \frac{1\,000\,000}{100} = 10\,000.00\text{ Hz}$$

$$\text{Duty} = \frac{50}{100} \times 100\% = 50.0\%$$

$$\text{err. rel.} = 0.0\%$$

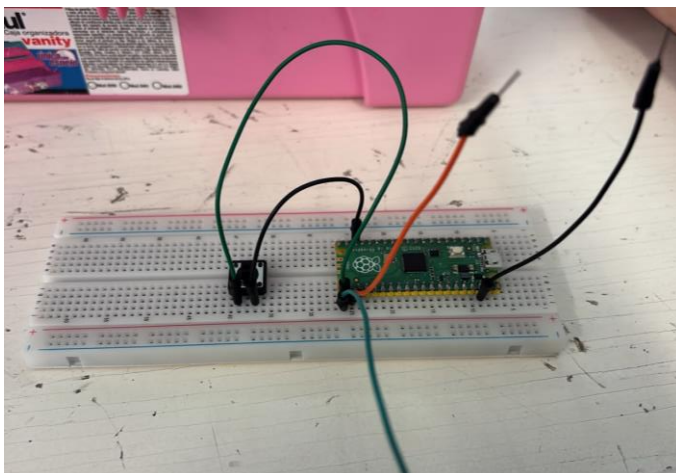
$$\text{err. duty} = 0.0\%$$

Tabla 1: Frecuencímetro

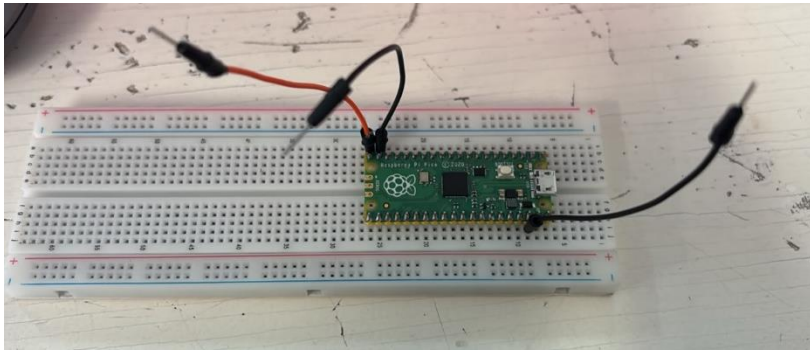
f referencia	f medida (μs)	err. rel.	duty medido	err. duty
100 Hz	100.00 Hz	0.0%	50.0%	0.0%
1 kHz	1000.00 Hz	0.0%	50.0%	0.0%
10 kHz	10000.00 Hz	0.0%	50.0%	0.0%

Evidencias

Antirrebote



Frecuencia



Conclusiones

En conclusión, esta práctica demostró la eficiencia de utilizar temporizadores de hardware e interrupciones para realizar mediciones temporales y gestionar entradas físicas de forma no bloqueante. Por un lado, la técnica de marcas de tiempo (*timestamping*) permitió determinar con precisión el periodo, la frecuencia y el *duty cycle* de una señal, evidenciando cómo el error de resolución impone un límite o techo a la frecuencia máxima medible. Por otro lado, la implementación del antirrebote por *timer* comprobó que es posible eliminar el ruido mecánico de los pulsadores sin detener el procesador con esperas activas.

Bibliografías

[1] M. N. Ramírez Cruz, «Bare-Metal I/O & Timing: HWtimers: compare/capture; timer-based debounces», material de clase de Sistemas Embebidos, Universidad Iberoamericana Puebla, Otoño 2026.